

Evaluation

Combine fast local checks with cloud-based Foundry evaluators

Source: <https://learn.microsoft.com/en-us/agent-framework/agents/evaluation?pivots=programming-language-python>

Local checks and Foundry evaluators complement each other

Local Evaluators: LocalEvaluator

- No evaluator API call
- Exact tool names and argument subsets
- Fast inner loop and CI smoke tests

Microsoft Foundry Evaluators: FoundryEvals

- Cloud-based evaluation service
- LLM-as-judge quality and agent behavior
- Portal dashboards and comparisons

Local evaluators: built-in checks

Check	What it does
<code>keyword_check(*keywords)</code>	Response must contain all specified keywords
<code>tool_called_check(*tool_names)</code>	Agent must have called the specified tools
<code>tool_calls_present</code>	All <code>expected_tool_calls</code> names appear in the conversation (unordered, extras OK)
<code>tool_call_args_match</code>	Expected tool calls match on name and arguments (subset match on args)

Check tool name and arguments locally

```
expected = ExpectedToolCall(
    "wit_work_item",
    {"action": "get", "id": 42},
)

local = LocalEvaluator(
    tool_calls_present,
    tool_call_args_match,
)

results = await evaluate_agent(
    agent=agent,
    queries=["Get work item 42."],
    expected_tool_calls=[expected],
    evaluators=local,
)
```

D E M O

~5 min

evaluate_agent catches a wrong tool call

Run the previous slide's exact `evaluate_agent` call against a seeded query the agent picks the wrong tool for: `tool_calls_present/tool_call_args_match` fail the check and print exactly which expectation didn't match — evaluation catching a real regression, not a synthetic pass.

Runbook: [demos/day3/module-7-demo-1-eval-catches-wrong-tool.md](#)

Custom evaluators wrap any function

```
from agent_framework import evaluator, LocalEvaluator

@evaluator
def is_concise(response: str) -> bool:
    """Check response is under 500 words."""
    return len(response.split()) < 500

@evaluator
def mentions_city(response: str, expected_output: str) -> bool:
    """Check response contains the expected city name."""
    return expected_output.lower() in response.lower()

@evaluator
def used_tools(conversation: list, tools: list) -> float:
    """Score based on tool usage. Returns 0.0-1.0 (>= 0.5 passes)."""
    tool_calls = [c for m in conversation for c in (m.contents or []) if c.type == "function_call"]
    return min(len(tool_calls) / max(len(tools), 1), 1.0)

local = LocalEvaluator(is_concise, mentions_city, used_tools)
```

Foundry evaluators

```
from agent_framework.foundry import FoundryEvals

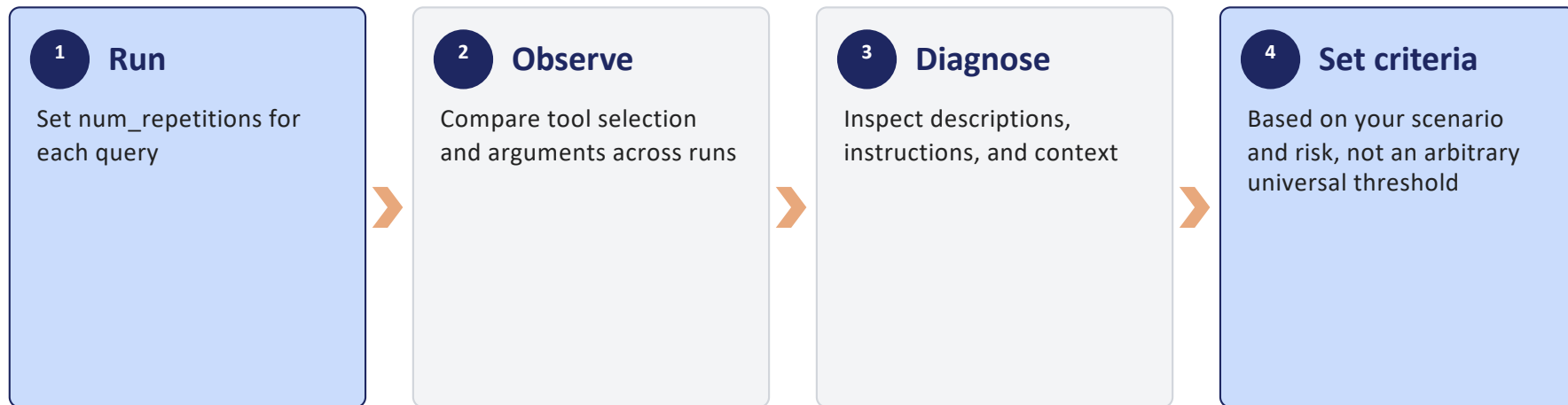
evals = FoundryEvals(
    project_client=project_client,
    model="gpt-4o",
    evaluators=[FoundryEvals.RELEVANCE, FoundryEvals.COHERENCE],
)
```

FoundryEvals connects to Microsoft Foundry's evaluation service for cloud-based LLM-as-judge evaluation. Results are viewable in the Foundry portal with dashboards and comparison views.

Available Foundry evaluators

Category	Evaluators
Agent behavior	intent_resolution, task_adherence, task_completion, task_navigation_efficiency
Tool usage	tool_call_accuracy, tool_selection, tool_input_accuracy, tool_output_utilization, tool_call_success
Quality	coherence, fluency, relevance, groundedness, response_completeness, similarity
Safety	violence, sexual, self_harm, hate_unfairness

Use Repetitions to handle nondeterminism



Takeaways

- ✓ You choose between LocalEvaluator (fast, deterministic, no API call) and FoundryEvals (cloud-based, LLM-as-judge), or combine both in one `evaluate_agent` call.
- ✓ You use the built-in local checks (`keyword_check`, `tool_called_check`, `tool_calls_present`, `tool_call_args_match`), or wrap your own function with the `@evaluator` decorator.
- ✓ You use `ExpectedToolCall` with LocalEvaluator to check exact tool names and arguments, as in the Azure DevOps example.
- ✓ FoundryEvals covers four evaluator categories: Agent behavior, Tool usage, Quality, and Safety.
- ✓ You use `num_repetitions` to expose variance across independent runs, not to justify a single universal pass threshold.